There are a couple of other Ruby ways to do something similar. Rather than create a subclass, you can create an instance for testing and add overriding methods to that instance's singleton class.

That class might be defined like this:

```
test_server = FlitServer.new
class << test_server
  def check_for_mentions(flit)
    # test code
  end
  def follower_list(user)
    # test code
  end
end
```

Alternatively, you could create a complete dummy class that covers the calls made by your method under test. Since Ruby doesn't do any type-checking beyond seeing whether the object responds to methods, that's perfectly fine:

```
class FakeServer
  def check_for_mentions(flit)
    # test code
  end
  def follower_list(user)
    # test code
  end
end
```

Now you need to inject the new object into the test code. In some sense you're reimplementing what a mock-object package would be doing. You can try to inject the object in the test itself by doing the same thing to the flit object that you did to the flit_server object. Here's an example:

```
Class TestFlit < Flit
  def flit_server
    TestFlitServer.new
  end
end

it "processes a flit correctly if it has followers" do
  user = User.create(screen_name: "zot")
  follower = user.followers.create(screen_name: "jennyw")
  flit = TestFlit.new(user, "Hello to $jennyw, How are things on earth?")
  flit.process_flit
  expect(follower.timeline.size).to eq(1)
end
```